

CS-317 Operating Systems



Lecture 9



Threads

Book 1 Chapter 4

In the last lecture...

- ▶ We looked at thread states.
- ▶ We were introduced to the idea of synchronization and its relation to threads.
- ▶ We realized how RPCs can benefit from multithreading.
- ▶ We explored user level threads.

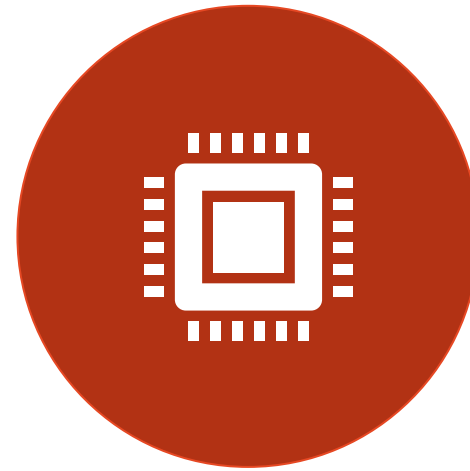
Types of threads

Different ways of implementation of threads

Types of threads



USER LEVEL THREADS
(ULTS)



KERNEL LEVEL THREADS
(KLTS)

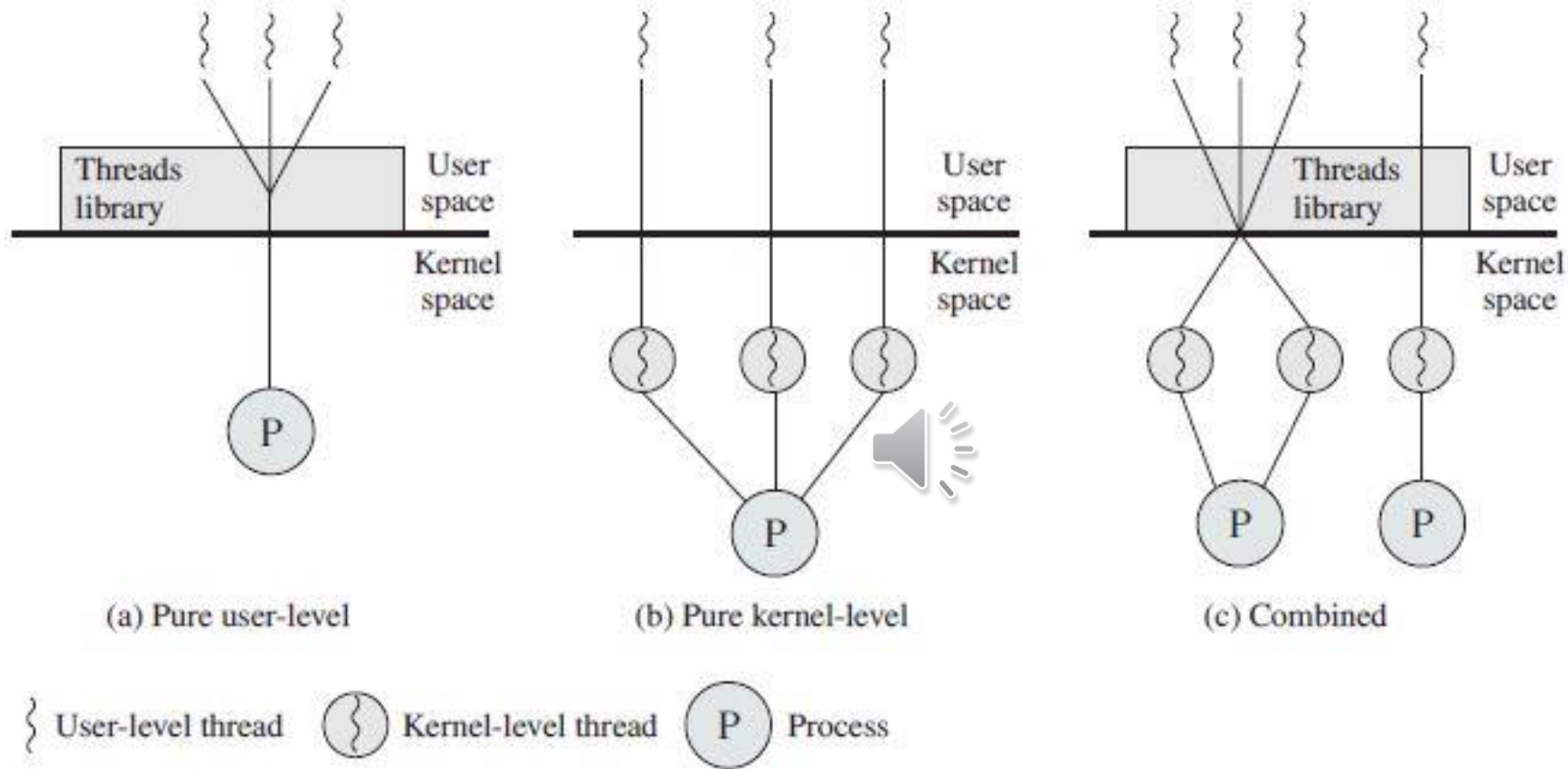
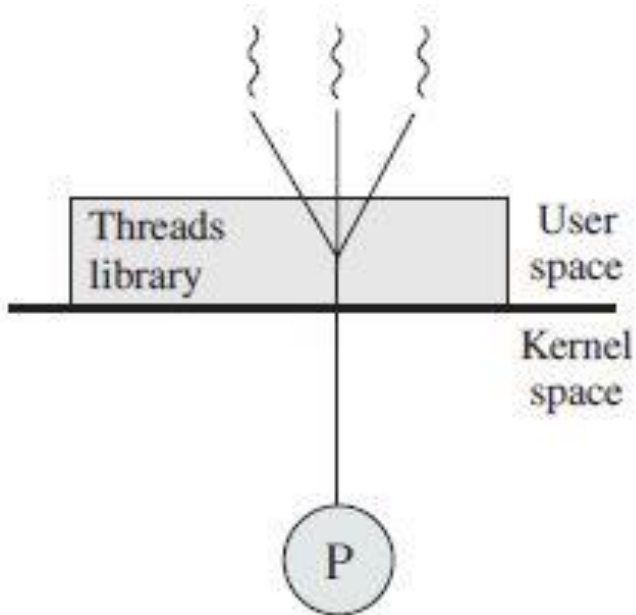


Figure 4.5 User-Level and Kernel-Level Threads

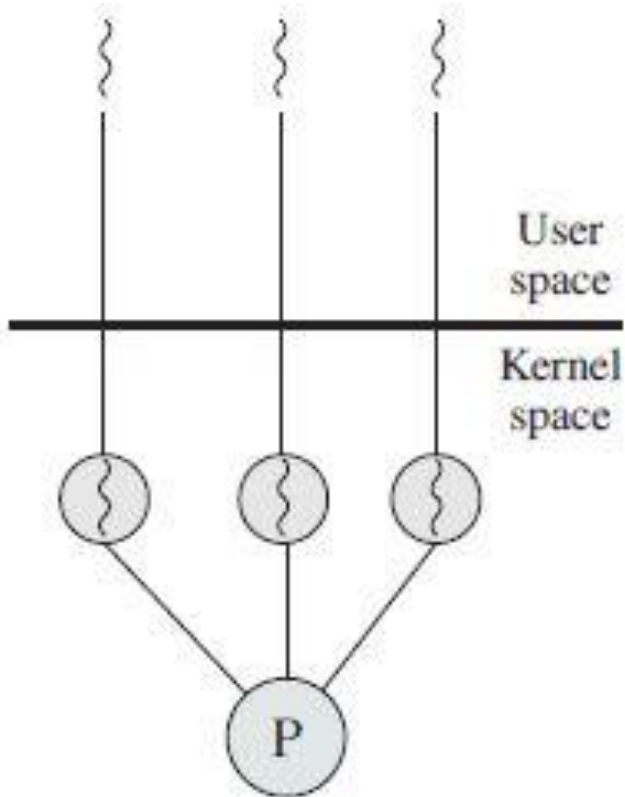
Approaches to thread implementation

User Level Threads Revision



- The OS does not know that ULTs exist because they are in the user space, managed by the threads library.
- When the OS schedules a process, the threads library schedules one of the ULTs of the process.
- We can run ULTs in any OS, schedule them using any algo we like, and we can switch between two ULTs of the same process without OS intervention.
- If one of these ULTs sends an OS service call, the OS thinks the process is making the OS service call, and the OS blocks the whole process.
- We can use the technique of jacketing to prevent a ULT from blocking, which will prevent our process from blocking.
- We cannot run two ULTs of the same process on different processors at the same time because the OS gives one process to one processor only.

Kernel Level Threads



- In a pure KLT facility, all the work of thread management is done by the kernel.
- There is no thread management code in the application level, simply an application programming interface (API) to the kernel thread facility.
- The kernel maintains context information for the process as a whole and for individual threads within the process.
- Scheduling by the kernel is done on a thread basis.

Advantages of KLTs



- ▶ When a KLT executes a system call, only this thread blocks. If a KLT blocks, another KLT within the same process can be selected.
- ▶ In a pure KLT strategy, a multithreaded application can take advantage of multiprocessing. The kernel can assign different threads of the same process to different processors at the same time.
- ▶ Kernel routines themselves can be multithreaded.

in ult, all threads gets blocked of that process

ult can only be done in uniprocessing

Disadvantages of KLTs

- ▶ Thread switching requires kernel mode privileges because all the thread management data structures are within the address space of the OS.
- ▶ Scheduling can not be application specific. The same scheduling algorithm is used by OS scheduler for all applications.
- ▶ KLTs can only run on an OS that supports threads.

ult does not kernel mode privileges,
because data structure in user space

in ult, scheduling is application specific

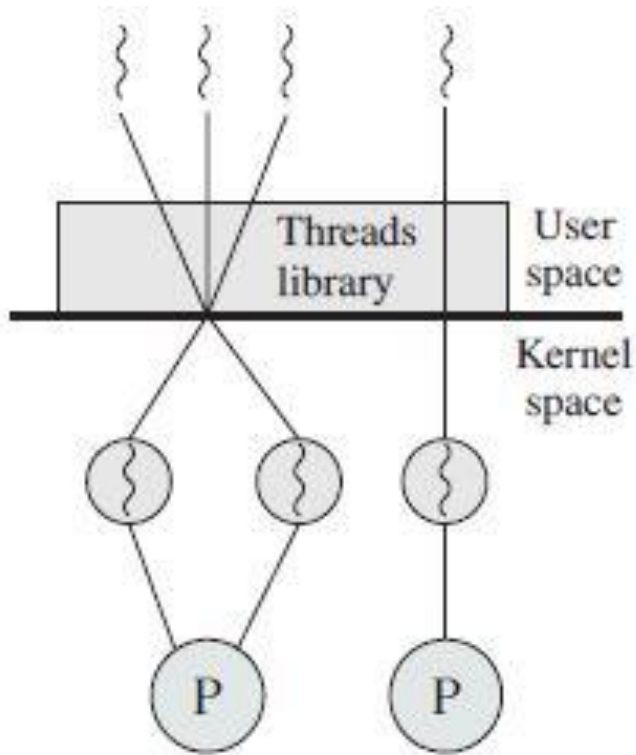
ult runs on os that does not support thread

Pause the video!

Write your answer in a word file. Save it as <my_roll_no>_lecture_9. If your roll no is 500, the file will be named 500_lecture_9. Do not add <> signs to the filename.

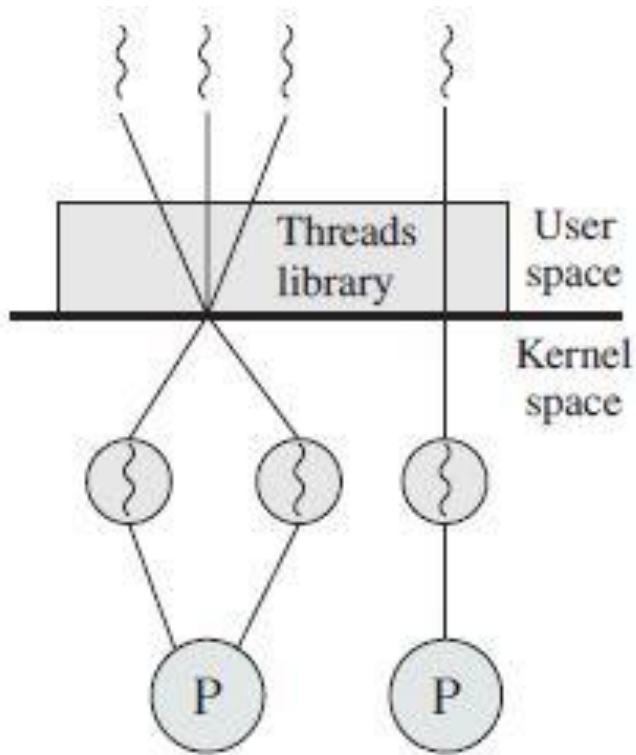
Task# 1: A process has three ULTs. All of them require OS services. Will implementing the process with KLTs improve performance? Why, or why not?

Combined Approach



- In a combined system, the threads library looks after:
 - thread creation,
 - scheduling, and
 - synchronization of threads within an application.
- Multiple ULTs from a single application are mapped onto some (smaller or equal) number of KLTs. , "single application" generally means a single process.
- The OS only sees the KLTs.
- When the OS schedules a KLT, the threads library runs one of the ULTs mapped onto this KLT.

Combined Approach



- Mapping many ULTs to a KLT is a good idea because, if one of them blocks, another can be mapped to the executing KLT. There is no need to switch the KLT.

KLT switch can happen even when both KLTs belong to the same process. since, each klt is treated as one unit

klt switch occurs in klt approach. also in ult, on one lt block whole process gets blocked

- The programmer may adjust the number of KLTs to achieve the best overall results.
- Multiple threads within the same application can run in parallel on multiple processors. this was not supported in ult
- A blocking system call need not block the entire process.

Pause the video!

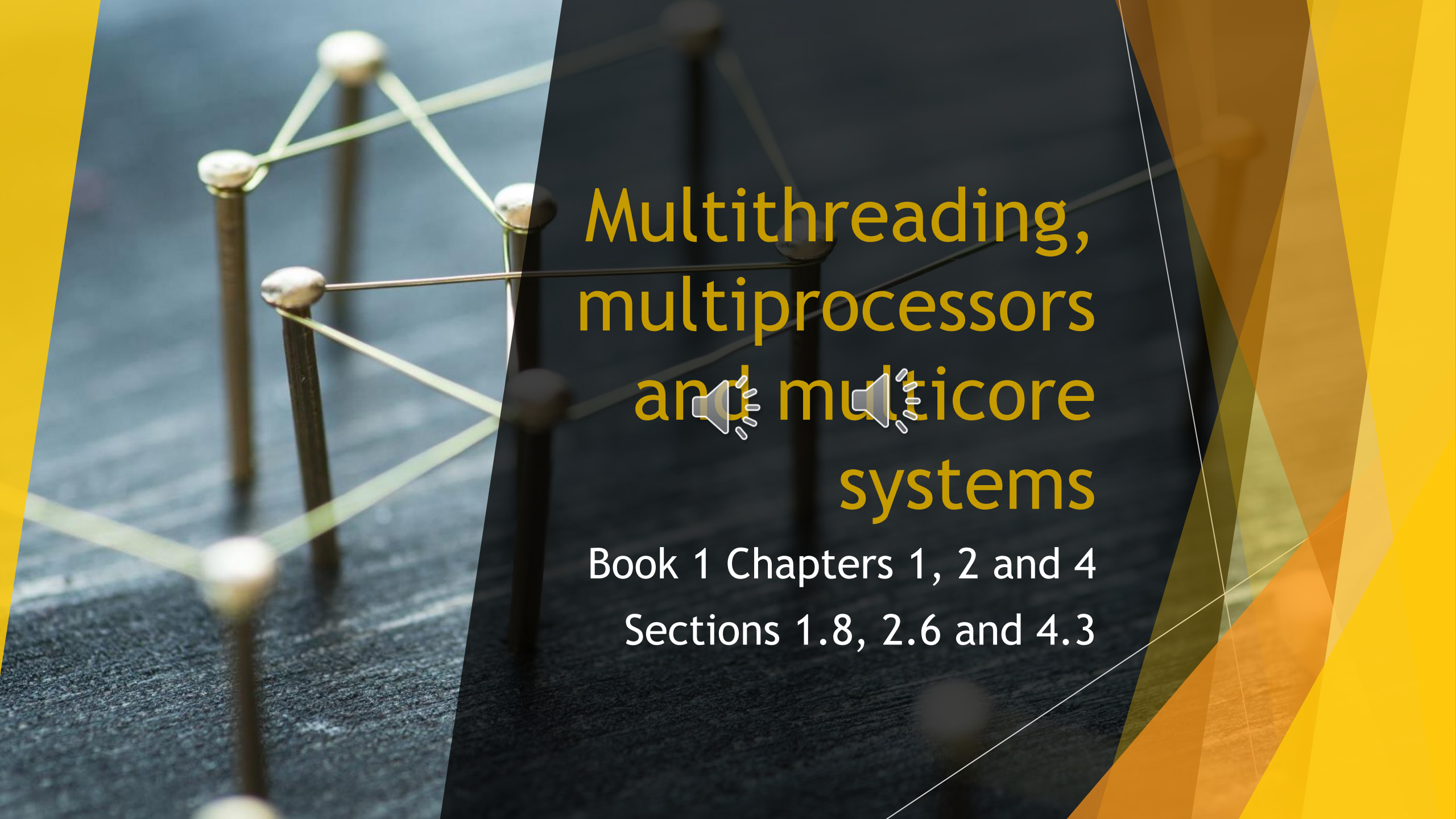
Write your answer in a word file previously saved as <my_roll_no>_lecture_9.

Task# 2: Mapping x ULTs on y KLTs in the combined approach is beneficial only when $x \geq y$. Comment.

Table 4.2 Relationship between Threads and Processes

Threads: Processes	Description	Example Systems
1:1	Each thread of execution is a unique process with its own address space and resources.	Traditional UNIX implementations
M:1	A process defines an address space and dynamic resource ownership. Multiple threads may be created and executed within that process.	Windows NT, Solaris, Linux, OS/2, OS/390, MACH
1:M	A thread may migrate from one process environment to another. This allows a thread to be easily moved among distinct systems.	Ra (Clouds), Emerald
M:N	Combines attributes of M:1 and 1:M cases.	TRIX

Thread-Process Arrangements



Multithreading, multiprocessors and multicore systems

Book 1 Chapters 1, 2 and 4
Sections 1.8, 2.6 and 4.3

Threads and multiprocessors



Multithreading and SMPs

Multiprocessors

- ▶ A computer which has many processors on the same motherboard is called a multiprocessor system.
- ▶ All processors share the resources that interface with the motherboard.
- ▶ All processors are managed by the same operating system.
- ▶ Based on execution of OS, multiprocessors can be categorized as:
 - ▶ Master/slave architecture,
 - ▶ Symmetric Multiprocessors (SMPs).

Master/slave architecture

- ▶ Only one processor in the system executes the operating system. This processor is called the master.
- ▶ All other processors execute user applications. They are called slaves.
- ▶ The operating system for master/slave architecture does not differ greatly from that designed for a uniprocessor system.
- ▶ The system has a performance bottleneck.
- ▶ The system has a single point of failure.



In a master-slave system:
The master controls the work and coordinates all slave nodes.
Slaves do the actual processing but depend on the master for instructions or data.
Bottleneck occurs because:
The master can become overloaded if there are many slaves.
All communication passes through the master, creating a single point of congestion.
If the master is slow, idle slaves must wait, reducing overall system performance

Symmetric Multiprocessors (SMPs)

- ▶ There are two or more similar processors of comparable capability.
- ▶ These processors share the same main memory and I/O facilities and are interconnected by a bus or other internal connection scheme, such that memory access time is approximately the same for each processor.
- ▶ All processors share access to I/O devices, either through the same channels or through different channels that provide paths to the same device.
- ▶ All processors can perform the same functions.
- ▶ The system is controlled by an integrated operating system that provides interaction between processors and their processes.
- ▶ Any processor in the system can execute the operating system. Kernel modules can be run simultaneously on more than one processors.

SMP Organization

- ▶ There are multiple processors, each containing its own control unit, arithmetic-logic unit, and registers.
- ▶ Each processor has access to a shared main memory and the I/O devices through some form of interconnection mechanism.

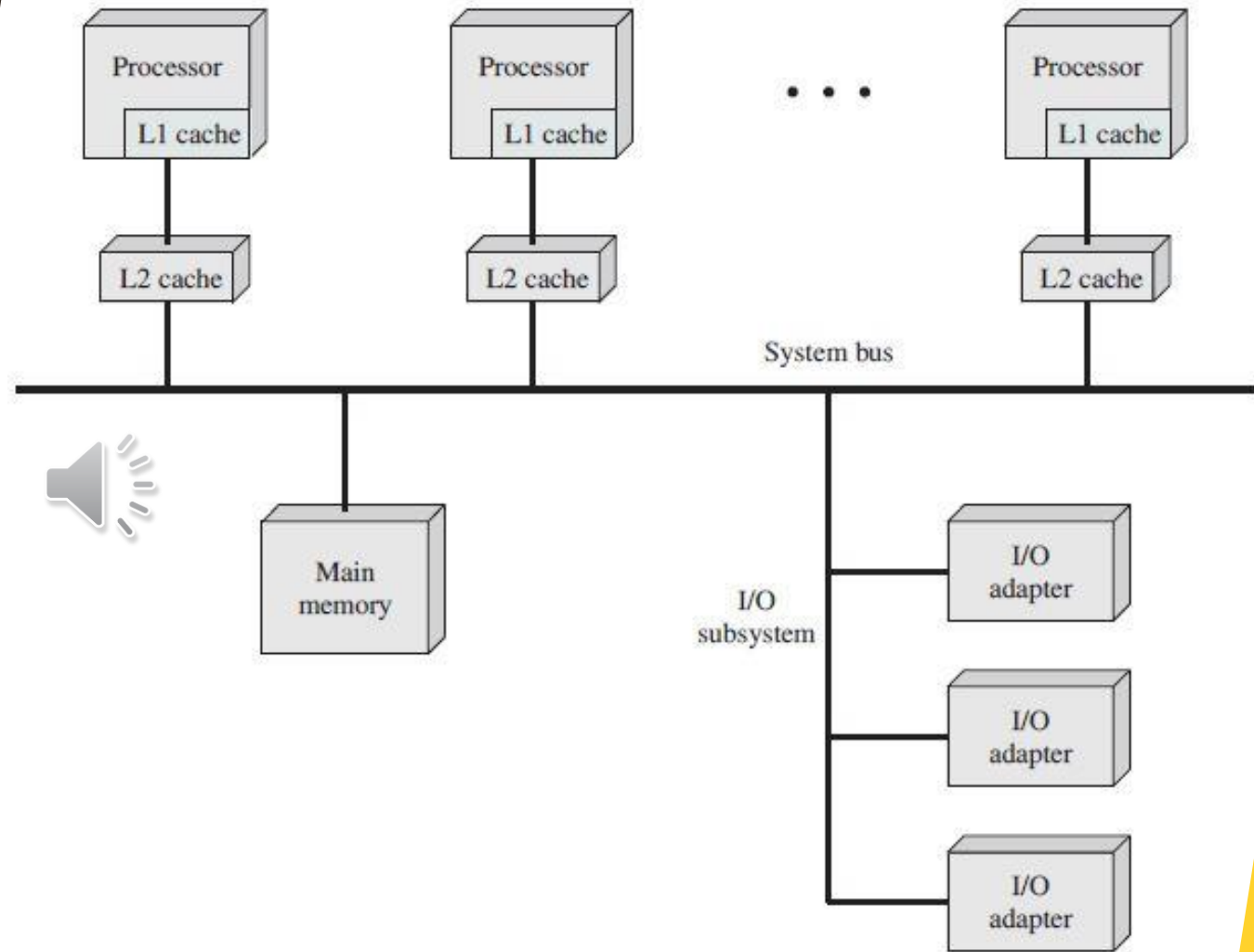


Figure 1.19 Symmetric Multiprocessor Organization

SMP Organization

- ▶ The processors can communicate with each other through shared memory.
- ▶ The memory is organized so that multiple simultaneous accesses to separate blocks of memory are possible.
- ▶ Processors have at least one level of cache memory that is private to them.

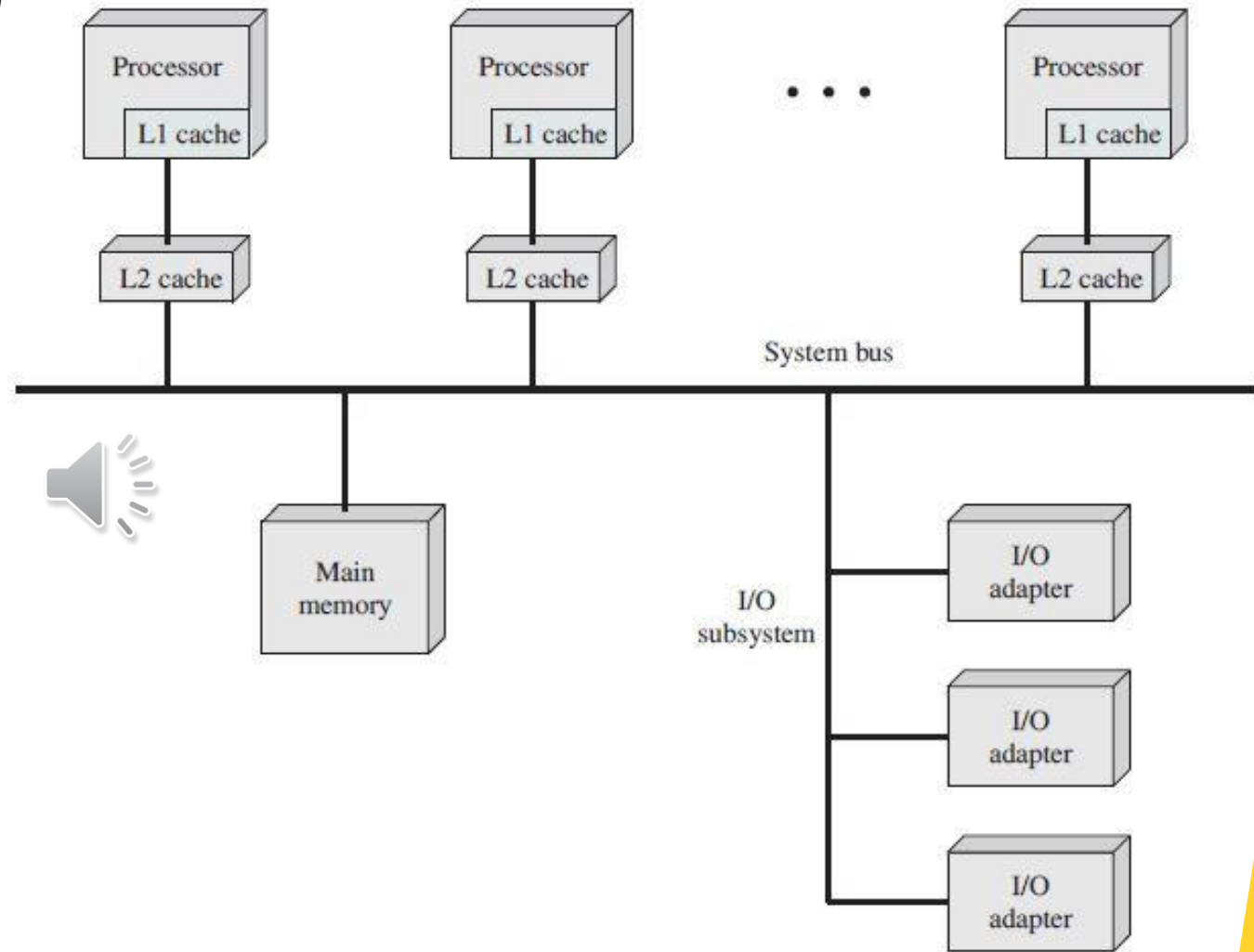


Figure 1.19 Symmetric Multiprocessor Organization

Potential benefits of SMPs



Performance: If portions of the work can be done in parallel, an SMP will yield greater performance than a uniprocessor.



Availability: In an SMP, the failure of a single processor does not halt the machine. Instead, the system continues to function at reduced performance.



Incremental growth: A user can enhance the performance of a system by adding an additional processor.



Scaling: Vendors can offer a range of products with different price and performance characteristics based on the number of processors configured in the system.

Role of OS in SMP Performance

- ▶ The benefits we saw just now are potential, rather than guaranteed, benefits.
- ▶ The operating system must provide tools and functions to exploit the parallelism in an SMP system.
- ▶ An attractive feature of an SMP is that the existence of multiple processors is transparent to the user.
- ▶ The operating system takes care of scheduling of tasks on individual processors and of synchronization among processors.

The user and applications do not need to know or manage how many processors are working. The OS handles everything behind the scenes

Multithreading and SMPs

- ▶ Multithreading and SMP are often discussed together, but the two are independent facilities.
- ▶ Even on a uniprocessor system, multithreading is useful for structuring applications and kernel processes.
- ▶ An SMP system is useful even for nonthreaded processes, because several processes can run in parallel.
- ▶ However, the two facilities complement each other and can be used effectively together.

Master-Slave Architecture → No (does not inherently support multithreading)

SMP (Symmetric Multiprocessing) → Yes (supports multithreading)

SMP OS Considerations



Design of an operating system for SMPs

Challenges involved



The OS designer must consider scenarios involving shared resources like data structures.



The OS designer must coordinate actions like simultaneous accesses of devices from multiple parts of the OS executing at the same time.



Techniques must be employed to resolve and synchronize claims to resources.



A multiprocessor OS must provide all the functionality of a multiprogramming system plus additional features to accommodate multiple processors.

Key issues in design of OS for SMPs



Simultaneous concurrent processes or threads



Scheduling



Synchronization



Memory management



Reliability and fault tolerance

Design issue# 1

Simultaneous concurrent processes or threads

- ▶ A computer program or routine is described as reentrant if it can be safely called again before its previous invocation has been completed. Reentrant means a program or function can be safely interrupted and called again (re-entered) before its previous execution is finished, without causing errors or incorrect results.
- ▶ Kernel routines need to be reentrant to allow several processors to execute the same kernel code simultaneously.
- ▶ With multiple processors executing the same or different parts of the kernel, kernel tables and management structures must be managed properly to avoid data corruption or invalid operations.

Finals Batch 2022:
why reentrant code is important with respect to smp

Reentrant code is important in Symmetric Multiprocessing (SMP) because multiple processors may execute the same code simultaneously.

Why it matters:

In SMP, different CPUs can run the same program/function at the same time. If the code is not reentrant, shared data or static variables may get corrupted due to concurrent access.

Reentrant code ensures:

No conflicts between processors

No data corruption

Safe concurrent execution

In short:

Reentrant code allows multiple processors in an SMP system to execute the same code at the same time without interfering with each other, making the system reliable and efficient.

Design issue# 2

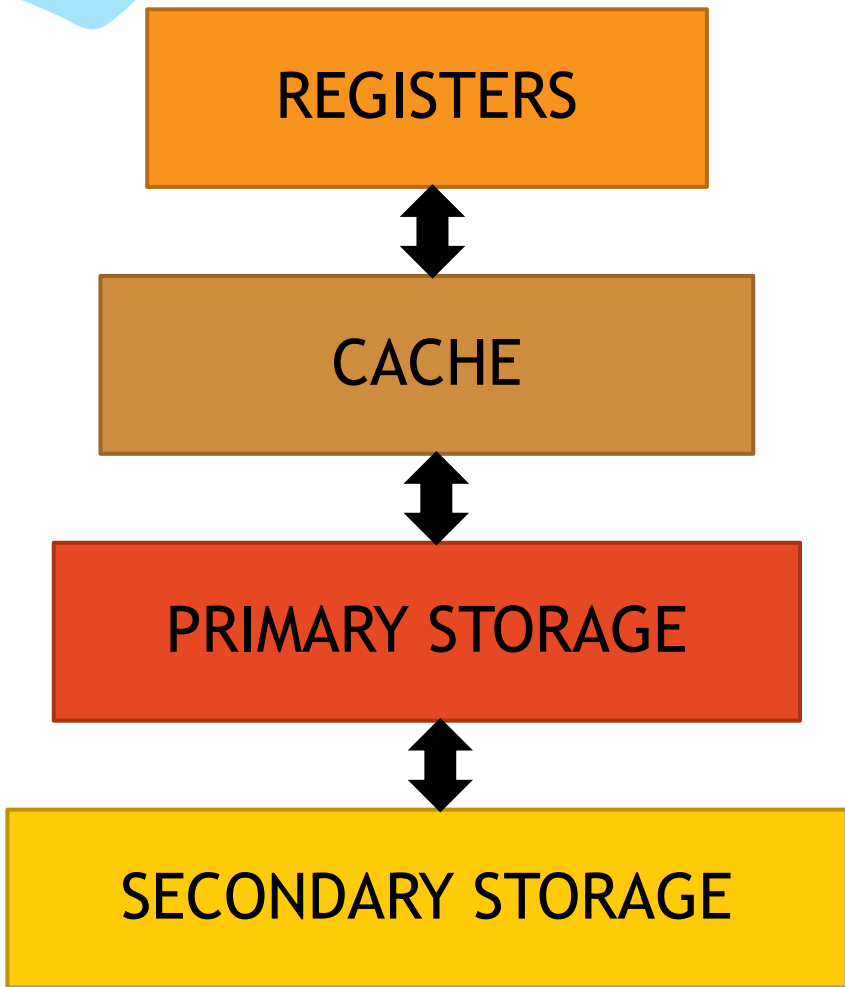
Scheduling

- ▶ Any processor may perform scheduling, which complicates the task of enforcing a scheduling policy and assuring that scheduler data structures do not corrupt.
- ▶ If kernel-level multithreading is used, then multiple threads from the same process can be scheduled simultaneously on multiple processors.

Design issue# 3

Synchronization

- ▶ With multiple active processes having access to shared memory or I/O resources, care must be taken to provide effective synchronization.
- ▶ Synchronization is a facility that enforces mutual exclusion and event ordering.



MEMORY HIERARCHY

Design issue# 4

Memory management

- ▶ Memory management on a multiprocessor must deal with all the issues of uniprocessors.
- ▶ The OS needs to exploit the available hardware parallelism.
- ▶ The paging mechanisms on different processors must be coordinated to enforce consistency when several processors share a page or segment and to decide on page replacement.
- ▶ If a page contains garbage or invalid data, it should be labelled appropriately so that no processor mistakes it for actual data.

Design issue# 5

Reliability and fault tolerance

- ▶ The OS should provide graceful degradation in the face of processor failure.
- ▶ The scheduler and other portions of the OS must recognize the loss of a processor and restructure management tables accordingly.

Lessons learned

- ▶ We explored kernel level threads.
- ▶ We explored the combined approach to threads.
- ▶ We talked in detail about symmetric multiprocessors.
- ▶ We realized that SMPs and multithreading strengthen each other.
- ▶ We discussed the design issues of an operating system for SMPs.

Things to do



Read the book.



I will ask you to submit the word file previously saved as <my_roll_no>_lecture_9 in Google classroom.



Write your questions on a piece of paper and keep it in front of you during the online session.



If you have suggestions relevant to content or quality of this lecture, email them to me.